```
  private record KeycloakToken(String accessToken) {
    @JsonCreator                                                    ❻
    private KeycloakToken(
      @JsonProperty("access_token") final String accessToken
    ) {
      this.accessToken = accessToken;
    }
  }
}
```

❶ A WebClient used to call Keycloak

❷ Authenticates as Isabelle and obtains an Access Token

❸ Authenticates as Bjorn and obtains an Access Token

❹ Uses the Password Grant flow to authenticate with Keycloak directly

❺ Blocks until a result is available. This is how we use WebClient imperatively rather than reactively.

❻ Instructs Jackson to use this constructor when deserializing JSON into KeycloakToken objects

Finally, we can update the test cases in `CatalogServiceApplicationTests` to cover several authentication and authorization scenarios. For example, we could write test cases for `POST` operations in these situations:

- The user is authenticated and has the `employee` role (extending the existing test case).
- The user is authenticated but doesn't have the `employee` role (new test case).
- The user is not authenticated (new test case).

NOTE In the context of an OAuth2 Resource Server, authentication means token authentication. In this case, it happens by providing an Access Token in the `Authorization` header of each HTTP request.

Create operations are only allowed to the bookshop employees, so only the first request will return a successful answer.

Listing 12.15 Verifying security scenarios in integration tests